

General Training for Programming Competitions & Coding Interviews

December 4, 2024

Winter semester 24/25

Markus Fischer, Paul G. Rump, Christoph Staudt



Minimum Spanning Trees

Spanning Trees: Definition

Let $G = (V, E)$ be a graph that is connected (that is G has only one component). A *spanning tree* of G is a subgraph $T = (V, F)$ with $F \subseteq E$ such that

- T is connected and
- T is a tree (contains no cycles).

Note that by definition, T has to contain *all* nodes from the original graph G .

Minimum Spanning Trees: Definition

Let $G = (V, E, w)$ be a connected graph with edge-weights, that is $w : E \mapsto \mathbb{R}$. A *minimum spanning tree* of G is a subgraph $T = (V, F)$ with $F \subseteq E$ such that

- T is connected, and
- T is a tree (contains no cycles), and
- there is no other spanning tree $T' = (V, F')$ with $\sum_{e \in F'} w(e) < \sum_{e \in F} w(e)$.

A minimum spanning tree is a spanning tree with minimal edge weights.

Note that a graph can have multiple different minimum spanning trees.

Motivation

- Find cheapest way to connect all points in a network,
e. g. planning supply of water-/electricity-/internet for a city district.
- Also often a basic building block of more complex algorithms.

Algorithms

There are two popular algorithms for construction of a minimum spanning tree:

- Prim's algorithm,
- Kruskal's algorithm.

They have different approaches, but are both very simple and greedy-ish.

Prim's Algorithm

The general idea of Prim's algorithm is to start with a single random node, and iteratively construct the minimum spanning tree by always adding an edge with minimum weight that connects a node from the current tree to a new node.

- Input: a weighted graph $G = (V, E, w)$
- Output: a minimal spanning tree of G .

Prim's Algorithm

The general idea of Prim's algorithm is to start with a single random node, and iteratively construct the minimum spanning tree by always adding an edge with minimum weight that connects a node from the current tree to a new node.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. Start with an arbitrary node $v \in V$, add v to W .

Prim's Algorithm

The general idea of Prim's algorithm is to start with a single random node, and iteratively construct the minimum spanning tree by always adding an edge with minimum weight that connects a node from the current tree to a new node.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. Start with an arbitrary node $v \in V$, add v to W .
 3. While $W \neq V$:
 - 3.1 Find an edge e that connects a node $w \in W$ with a node $v' \in V \setminus W$ with minimal weight.
 - 3.2 Add v' to W and add e to F .
 4. Return $T = (V, F)$.

Prim's Algorithm

The general idea of Prim's algorithm is to start with a single random node, and iteratively construct the minimum spanning tree by always adding an edge with minimum weight that connects a node from the current tree to a new node.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. Start with an arbitrary node $v \in V$, add v to W .
 3. While $W \neq V$:
 - 3.1 Find an edge e that connects a node $w \in W$ with a node $v' \in V \setminus W$ with minimal weight.
 - 3.2 Add v' to W and add e to F .
 4. Return $T = (V, F)$.
- Complexity: $\mathcal{O}(|E| + |V| \log(|V|))$ (looking at all edges + priority queue of all nodes).

Kruskal's Algorithm

Kruskal's Algorithm constructs a minimum spanning tree by iteratively adding the edge with minimum weight that does not construct a cycle.

- Input: a weighted graph $G = (V, E, w)$
- Output: a minimal spanning tree of G .

Kruskal's Algorithm

Kruskal's Algorithm constructs a minimum spanning tree by iteratively adding the edge with minimum weight that does not construct a cycle.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. $L =$ all edges, sorted by their weight (increasing order).

Kruskal's Algorithm

Kruskal's Algorithm constructs a minimum spanning tree by iteratively adding the edge with minimum weight that does not construct a cycle.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. $L =$ all edges, sorted by their weight (increasing order).
 3. For $e = \{v_1, v_2\}$ in L :
 - 3.1 If $T' = (W \cup \{v_1, v_2\}, F \cup e)$ does not contain a cycle:
Set $W = W \cup \{v_1, v_2\}$ and $F = F \cup e$.
 4. Return $T = (V, F)$.

Kruskal's Algorithm

Kruskal's Algorithm constructs a minimum spanning tree by iteratively adding the edge with minimum weight that does not construct a cycle.

- Input: a weighted graph $G = (V, E, w)$
 - Output: a minimal spanning tree of G .
1. Initialize $F = \emptyset$, $W = \emptyset$
 2. $L =$ all edges, sorted by their weight (increasing order).
 3. For $e = \{v_1, v_2\}$ in L :
 - 3.1 If $T' = (W \cup \{v_1, v_2\}, F \cup e)$ does not contain a cycle:
Set $W = W \cup \{v_1, v_2\}$ and $F = F \cup e$.
 4. Return $T = (V, F)$.
- Complexity: $\approx \mathcal{O}(|E| \log(|E|))$ (for sorting the edges, plus time for checking for cycles).

Checking for Cycles: Union-Find

Kruskal's Algorithm requires that we check if our current subgraph $T = (W, F)$ will contain a cycle if we add an edge $e = \{v_1, v_2\}$. This can be done by an *union-find-structure*.

- We want a data-structure that can for a given node *find* its component (if an edge connects two nodes that belong to the same component, it will introduce a cycle).
- Also we have to be able to *union* two components (if an edge connects two nodes of different components, both components become one by adding the edge).

Union-Find-Structure: Implementation

Trivial implementation: Array of length $|V|$ that contains for each node some identifier of its component.

- Find: $\mathcal{O}(1)$
- Union: $\mathcal{O}(|V|)$

Union-Find-Structure: Implementation

Trivial implementation: Array of length $|V|$ that contains for each node some identifier of its component.

- Find: $\mathcal{O}(1)$
- Union: $\mathcal{O}(|V|)$

Better implementation: For each component, build a tree that contains all nodes of the component

- Find (Search through all trees): $\mathcal{O}(|V|)$
- Union (add the root of one tree as child to the root of the other): $\mathcal{O}(1)$

However, there are heuristics that reduce the complexity of the find()-operation to $\mathcal{O}(\log(|V|))$ (not discussed here).

Union-Find-Structure in Kruskal's Algorithm

During Kruskal's Algorithm, we need:

- `find()`: $2|E|$ times (for each edge, call `find()` two times (to check if the edge introduces a cycle));
- `union()`: $|V| - 1$ times (each time we add an edge to T).

Union-Find-Structure in Kruskal's Algorithm

During Kruskal's Algorithm, we need:

- `find()`: $2|E|$ times (for each edge, call `find()` two times (to check if the edge introduces a cycle));
- `union()`: $|V| - 1$ times (each time we add an edge to T).

Therefore in total during Kruskal's algorithm

- the trivial array-based implementation would cost $\mathcal{O}(|E| + |V|^2)$,
- and an optimized tree-based implementation would cost $\mathcal{O}(|E| \log(|V|))$.

Topological Sorting

Topological Sorting: Definition

Given an directed acyclic graph (DAG) $G = (V, E)$, a *topological sort* is a linear ordering of the nodes $\{v_1, \dots, v_n\} = V$ such that:

- v_i comes before v_j in the ordering if there is a (directed) path from v_i to v_j .
- Or, in math language, it is a *partial order* of V defined by:
 $\exists$ a (directed) path from v_i to $v_j \Rightarrow v_i \leq v_j$.

Topological orders can be used to schedule a list of tasks with lots of dependencies.

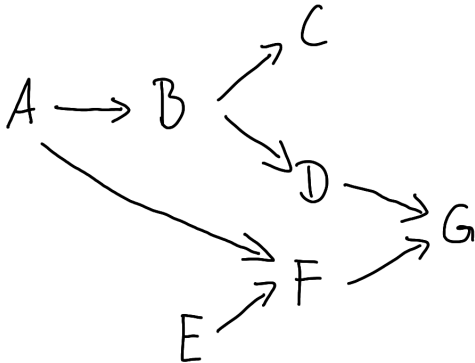
Topological Sorting: Example

Consider the tasks A, B, C, D, E, F, G with the following dependencies:

- A needs to be done before B ,
- finishing B is required to start C or D ,
- A and E have to be done before F , and
- D and F have to be done before G .

We can construct a directed graph from these, where the tasks are nodes and the dependencies are directed edges. By computing a topological sort on that graph, we get a valid order in which we can perform the tasks.

Topological Sorting: Example



valid order:

A, B, C, D, E, F, G or

E, A, F, B, D, G, C

invalid order:

A, B, C, D, **G**, E, F or

E, **F**, **G**, A, B, C, D

Algorithm

We can construct a topological sort with a (slightly modified) *depth-first-search*. The idea is

Algorithm

We can construct a topological sort with a (slightly modified) *depth-first-search*. The idea is to find nodes with no outgoing edges to unvisited nodes, these are valid **last** nodes of the ordering.

Algorithm

We can construct a topological sort with a (slightly modified) *depth-first-search*. The idea is to find nodes with no outgoing edges to unvisited nodes, these are valid **last** nodes of the ordering.

- Input: a DAG $G = (V, E)$.
 - Output: a topological ordering of the nodes V .
1. Initialize $L = []$
 2. While $\exists v \in V$ with $v \notin L$:
 - Start DFS at v . Whenever a node v' is found that has no outgoing edge to an unvisited node, add v' to L .
 3. Reverse L .
 4. Return L .

Algorithm

We can construct a topological sort with a (slightly modified) *depth-first-search*. The idea is to find nodes with no outgoing edges to unvisited nodes, these are valid **last** nodes of the ordering.

- Input: a DAG $G = (V, E)$.
 - Output: a topological ordering of the nodes V .
1. Initialize $L = []$
 2. While $\exists v \in V$ with $v \notin L$:
 - Start DFS at v . Whenever a node v' is found that has no outgoing edge to an unvisited node, add v' to L .
 3. Reverse L .
 4. Return L .
- Complexity: $\mathcal{O}(|V| + |E|)$ (for the DFS).

Exercise

Exercise

- Project Euler Problem 1: <https://projecteuler.net/problem=1> and
- Project Euler Problem 107: <https://projecteuler.net/problem=107>.

For each Exercise:

- Develop an algorithm that solves the problem (in C++ or Python).
- Upload your code to your repository (or include your code to your submission at the end of the semester).
- Describe your approach (see exam questions).

About Project Euler:

- Website with math & programming problems.
- Most problems require some coding to solve.
- Problems are stated in a programming-contest-style, however there are no rankings or leaderboards.
- If you register an account (free), you can check if your solution of a problem is correct (no evaluation of runtime, memory usage, etc.).

Exam Questions

Exam Questions

1. Argue why topological sortings only exist for **acyclic** graphs.
2. Briefly describe how you approached and solved Project Euler Problem 1.
3. Describe your approach on Project Euler Problem 107: which algorithm did you use and why?